Retinal Screening and Referral Support: Final Product Technical Report

Michael Kimani · BSc Software Engineering, African Leadership University · Supervisor: Emmanuel Adjei · 29 October 2026 Repository: https://github.com/Mikekimm/michael-kimani-eye-screening-capstone ·
Video walkthrough: https://youtu.be/t8ew6KO18XU

What this is, and what it is not. This is a screening and referral support tool, not a diagnostic device. The models were trained and tested only on public datasets. There has been no field deployment, no external validation and no clinical validation. Every number in this report comes from a recorded run saved in the repository (the docs/ and notebooks/ folders).

1. Scope alignment and feature validation

The approved proposal described an offline-first Android app that screens fundus photos for diabetic retinopathy (DR), glaucoma and cataract on the phone, refers uncertain or high-risk cases to a specialist dashboard, and syncs when a connection returns. After my supervisor's feedback (build one demonstrable workflow, reconcile offline use with sync and sign-in, keep the research evaluation separate from development), I put my effort into one complete, tested workflow first. Because the app loads each model from its own folder, adding glaucoma and cataract afterwards meant training models and dropping in their files, not rewriting code.

The biggest change from the proposal is the platform. Instead of React Native with on-device TensorFlow Lite, I built an installable web app (a PWA) that scores photos on the server. One code base then serves Android phones and iPhones, and models can be checked and replaced in one place. The cost is that offline mode can capture and queue photos, but cannot score them until the phone reconnects. The line-by-line comparison with the proposal's objectives and research questions is in docs/scope_table.md.

Planned feature	Intended purpose	Status	Evidence
DR classifier (MobileNetV2)	Flag signs of diabetic retinopathy	**Completed**	v2.0 served. On 35,426 test images: sensitivity 0.851 (target met), specificity 0.748 (target not met)
Glaucoma classifier (ODIR-5K)	Flag possible glaucoma	**Completed, below target**	Sensitivity 0.762, specificity 0.721, precision 0.103 (1,040 eyes, 42 positive)
Cataract classifier (ODIR-5K)	Flag possible cataract	**Completed, limited evidence**	Sensitivity 0.978, specificity 0.962, precision 0.542; only 46 positive test eyes
Severity scale 1 to 5	Show how serious a finding may be	**Changed**	DR: an estimated band from the model score, not a clinical grade. Five-band cataract grading not implemented, because ODIR-5K has no severity labels
Targets: sensitivity ≥ 85%, specificity ≥ 80%	Judge whether a model is good enough to screen	**Partial**	DR meets sensitivity only; glaucoma meets neither; cataract meets both (small sample)
On-device inference	Screen with no connection	**Changed (not met)**	Scoring runs on the server (LiteRT). Offline means capture and queue, scored after sync
Android app in React Native	Reach health workers' phones	**Changed**	Installable PWA for Android and iPhone, tested on Chromium, Firefox and WebKit
Offline storage and two way sync	Keep working with poor connectivity	**Completed**	IndexedDB queue, retry-safe uploads, decisions pulled back; tested with lost replies and disconnects mid-queue
Register patient, capture, screen, view result	The core health-worker workflow	**Completed**	End-to-end tests on four browser/device profiles. Patients are stored as a code and year of birth, with no name

Flag uncertain or high risk cases	Make sure doubtful cases reach a specialist	**Completed**	Borderline or low-quality cases are queued automatically; clear positives are sent by the health worker

Specialist review: approve or override	A specialist confirms or corrects the result	**Completed**	Shared queue; approve, override with a reason, request a retake, comment; the decision returns to the health worker
Admin: accounts and analytics	Control who uses the system and see usage	**Completed**	Sign-up requests approved or rejected by an administrator; roles; aggregate analytics
FastAPI + PostgreSQL backend	Store records and serve the app	**Completed**	SQLite locally; backend tests run on PostgreSQL 16 in GitHub Actions
Load balancer	Serve many clinics at once	**Not implemented**	The API is stateless, so it could sit behind one

Planned feature	Intended purpose	Status	Evidence
Usability study with health workers	Check real people can use it	**Partial (indicative only)**	SUS kit and results sheet in docs/usability. Remote sessions with a few testers who are not health workers: [ADD: number of testers, SUS mean, task success]
UML design	Specify the system before building it	**Completed**	Use case, class, ERD, sequence and architecture diagrams (docs/diagrams)
Added beyond the proposal	Safer, clearer clinical use	**Extension**	Specialty routing, a fundus check that refuses non-eye photos, a SHA-256-verified model registry, an append-only review log and an audit log

2. Technical functionality



Figure 1. Architecture as implemented (generated from docs/diagrams/src/architecture.mmd).

Front end. A React 18 and TypeScript PWA, designed for phones first (bottom tabs on small screens, a sidebar on large ones). Pages never call the server directly: they use hooks built on one typed API client, which attaches the sign-in token, refreshes it once when it expires, and turns errors into plain-language messages. A health worker registers a patient by clinic code, chooses the eye, and photographs it through a circular guide (rear camera or gallery). The app warns about dark or blurry photos before upload.

Back end and screening. FastAPI in three layers: routers (HTTP and role checks), services (the use cases) and repositories (all database access). When a photo arrives, the server checks the file type and size, confirms the bytes really decode as JPEG or PNG, applies a quality gate and a fundus check (blank, tiny or non-eye photos are refused with a reason and nothing is saved), and then runs every model the registry has verified. Each score goes through

that condition's risk policy. One result row is stored per condition with its model version, including a "not available" row if a model is missing. Each result card shows that model's measured limits, and results from an earlier model are never re-scored.

Referral workflow. A referral advisor turns the results into one recommendation. As in the proposal's sequence diagram, borderline results and usable photos with quality warnings are flagged and queued for a specialist automatically (this is configurable). A clear positive shows "Refer recommended" and the health worker sends it with "Ask a specialist"; clear negatives are not queued. Severe results (DR band 4 or 5) default to urgent. Review is asynchronous: cases wait in a shared queue with a review target (urgent 24 hours, routine 72 hours, my own design choice) and overdue cases rise to the top. A specialist can approve, override any condition (with a reason), request a retake, or comment. Each action is added to an append-only log that the ORM refuses to edit or delete, and it reaches the health worker on the next sync. The review confirms or corrects the result afterwards; it is not a gate, because the health worker still gives the recommended action at the visit. Beyond the proposal, specialists have specialties (DR, glaucoma, cataract, general), the queue shows matching cases first, and "Take this case" tells others it is taken. Specialists do not prescribe, and SMS or email alerts are future work.

Offline and sync. Without a connection, photos and new patients are saved in the browser's IndexedDB, each with a client-generated ID. The sync engine uploads them in order (patients first) and retries with exponential backoff and random jitter. Because the server recognises that ID, a retry after a lost reply returns the original record instead of creating a duplicate. Each cycle then pulls any specialist decisions made since the last one. Offline mode captures and queues; it does not run a model on the device.

Access control and privacy. Sign-up is a request: name, email, password and the role wanted (health worker or specialist; admin cannot be requested). Until an administrator approves it, a correct password only returns "waiting for administrator approval", and the requested role grants nothing. A second sign-up with a registered email is refused and changes nothing. The app does not verify anyone's identity or qualifications; the administrator is expected to check that outside the app. Every request is checked on the server, users cannot change their own role, sign-in and sign-up are rate limited, and passwords use Argon2id with rotating refresh tokens. Patients are pseudonymous codes, images are stored under random names and shown only to authorised users, analytics and audit records contain no images or patient fields, and secrets live in a git-ignored .env file.

3. Code quality, OOP and algorithms

The code is split by responsibility (routers, services, repositories, presenters and schemas) instead of living in one script. My earlier Streamlit prototype was removed and tagged in git, not left beside the new code.

Principle or pattern	Where it appears
Abstraction and polymorphism	ConditionModel with TFLiteConditionModel; tests swap in a clearly labelled test double
Strategy	A RiskPolicy per condition, built from JSON by RiskPolicyFactory
Open/closed	ModelRegistry: a new condition is a new folder holding model.tflite and model_card.json
Repository and dependency injection	Backend repositories; the front end's QueueRepository (IndexedDB or in-memory); api/deps.py
Encapsulation	Append-only history enforced by ORM listeners; one AccessPolicy decides who can see a record

Algorithm	Purpose and cost
Laplacian variance, brightness and vignette checks	Quality gate and refusal of non-eye photos; O(pixels) on a 256 × 256 copy
Sensitivity-targeted threshold search	Finds the highest threshold that still reaches 85% sensitivity; O(n log n) by sorting and binary search
Rank-sum AUC and stratified bootstrap intervals	Honest evaluation using NumPy only; the AUC matches scikit-learn
Threshold-anchored severity bands	Bands can never contradict the decision (tested across a sweep of 1,001 scores)

Idempotent sync with backoff and full jitter	No duplicates and no retry storms
Streaming SHA-256 and per-model resize	Model integrity; each model receives the input it was trained with

Other practices: type hints throughout the backend; TypeScript in strict mode with ESLint (which forbids calling fetch outside the API client); Pydantic schemas kept separate from database entities; central configuration (settings, risk policy, model cards); pinned requirements and package-lock.json; and small conventional commits (129 on main, built
up from feature branches).

4. Testing approaches and results

Strategy	Why chosen	How	Result
Unit (algorithms)	A boundary error would silently mis-triage patients	pytest and Vitest with boundary values and a sweep test. I also broke the code on purpose to confirm the tests fail	302 backend tests (unit and integration) + 86 front-end tests pass
Integration (API, database, real models)	Security and integrity rules live between layers	Fresh database per test, real checksum-verified models, a 60-case role matrix, PostgreSQL in CI	Included in the 302 backend tests; PostgreSQL 16 in CI
End-to-end	Prove users can finish clinical tasks	Playwright: sign-up requests, capture with a refused non-eye photo, ask a specialist, review, decision and retake returned, unauthorised access, invalid input	51 passed, 3 skipped by design, 0 failed (4 profiles)
Compatibility	Clinics use varied phones and browsers	The same journeys on Chromium (Pixel 5 and desktop), Firefox and WebKit (iPhone 13)	All pass
Usability (remote, SUS)	Check that people other than the developer can finish the core tasks	Three tasks (sign up and sign in, screen a patient, find the result in History), then the 10-question SUS questionnaire; scored by scripts/sus_score.py	[ADD: testers, SUS mean, success, main problems]
Accessibility	Low-vision and screen reader users	axe-core, WCAG 2.1 A/AA, 20 pages, all roles, mobile and desktop	0 violations
Performance	Slow hardware and networks	Benchmark script, throttled browser round trip, Lighthouse	See section 5
Security (brute force)	Password guessing	Sliding-window login throttle; unit and API tests	Blocked (429) after 5 failures
Model validation and parity	The app must feed models what they were trained on	Held-out test splits; metrics recomputed with the app's own code; a 24-photo parity check	See section 6

Different inputs. I tried negative, borderline and positive scores; real and synthetic fundus photos; non-eye images; blank, tiny, corrupt, mislabelled, oversized and future-dated uploads; invalid patient data; weak passwords; expired and replayed tokens; and every role against every route. The fundus check wrongly refused 3 of 25 real fundus

photos and accepted none of 77 non-eye images. I tuned it on those same images and tested no face photos, so these figures are optimistic.

What testing changed:

• My supervisor’s live test showed a screenshot being accepted as an eye, and every positive result being sent automatically. The fundus check became a hard stop, and positives are now sent with “Ask a specialist”. • A form test showed the password field dropped its form reference, so sign-in could never succeed. Fixed with forwardRef.

• An end-to-end script test showed the calibration script did not write its outputs. Fixed.

• Lighthouse SEO rose from 91 to 100. axe-core found unlabelled camera inputs and, after the new models, low contrast amber badges. Both fixed.

• The parity test showed the app was resizing every photo with nearest-neighbour, which would have changed 6 of 24 DR v2 decisions. Each model card now states its own resize method.

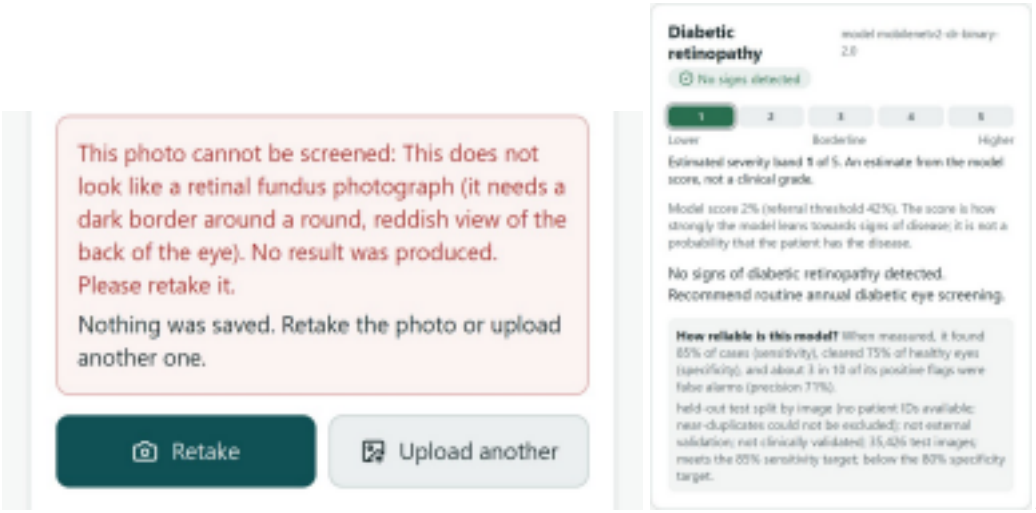


Figure 2. Left: the DR result card on an iPhone 13 profile (WebKit), from a synthetic test image, showing the estimated band, model version, score explanation and action. Right: a non-eye photo is refused and nothing is saved.

5. Performance and compatibility

Measurement (Windows 11, Intel CPU, 8 logical cores)	Result
Model inference, LiteRT, 200 runs (all cores / 1 thread), DR v1 2.6 MB model	55 / 60 ms mean; 68 / 84 ms p95; about 117 MB process memory (not yet re-run with the three 9 MB models)
Full server pipeline on a 1024 px JPEG (all cores / 1 thread)	74 / 77 ms mean (166 ms in an earlier run under background load)
Upload to result in the browser (median of 3)	0.60 s; 4.15 s with an emulated 6× slower CPU and slow 3G (three models)
Lighthouse, sign-in page (performance / accessibility / best practices / SEO)	Mobile 96 / 100 / 100 / 100; desktop 100 / 100 / 100 / 100
Browsers and devices (Playwright)	Chromium on Pixel 5 and desktop, Firefox desktop, WebKit on iPhone 13: 51 passed, 3 skipped by design (axe and throttling run on Chromium only), 0 failed
Bundle size	Route-level code splitting; core about 110 KB gzipped; charts (103 KB) load only for admins

Inference is a small part of the round trip, and limiting it to one thread costs only about 5 ms, so a modest server CPU is enough. On a slow phone and network the wait rises to about 4 seconds, mostly the upload; because of the offline queue, the worker can carry on with the next patient meanwhile. The low-end figures are emulated in Chrome DevTools. I have not tested a physical low-end phone. One real phone, an iPhone 13 Pro (iOS Safari), opened the app over a public HTTPS link and it worked in a short informal check (tasks were not timed).

6. Model evaluation, limitations and next steps

Model (test split)	Threshold	Sensitivity (95% CI)	Specificity (95% CI)	Precision	AUC	Targets 85% / 80%

DR v2.0 (35,426 images)	0.418	0.851 (0.845–0.857)	0.748 (0.743–0.754)	0.706	0.900	Sensitivity met, specificity not

Model (test split)	Threshold	Sensitivity (95% CI)	Specificity (95% CI)	Precision	AUC	Targets 85% / 80%
Glaucoma (1,040 eyes, 42 positive)	0.320	0.762	0.721	0.103	0.818	Not met
Cataract (1,044 eyes, 46 positive)	0.541	0.978	0.962	0.542	0.991	Met (limited evidence)

DR v2.0 is MobileNetV2 fine-tuned from block_13_expand on the Kaggle EyePACS + APTOS + Messidor collection, split 70/15/15 by image group. I chose the threshold on the validation split (the highest one with sensitivity of at least 85%) and scored the test split once. I recomputed every figure from the notebook's predictions using the app's own threshold and bootstrap code, and they matched. The served file has the notebook's checksum, and on 24 test photos the app reproduces the notebook's scores within 0.021 (my tolerance was 0.03) with no decision changed. Glaucoma and cataract (ODIR-5K) were split by patient, but ODIR labels come from free-text keywords and their positive classes are small. For reference only, Gulshan et al. (2016) reported 90.3% sensitivity and 98.1% specificity on different data and at a high-specificity operating point, so the comparison is not like-for-like. Our sensitivity for referable DR (grade 2 or worse) is 0.869. My earlier DR model (v0.3) reached only 0.542 sensitivity on its validation fold.

- **False alarms are real.** About a quarter of eyes without DR are referred (specificity 0.748), and about 9 in 10 glaucoma flags are false alarms. The result screens say so, and a specialist reviews every referral. • **The DR figures may be optimistic.** The dataset has no patient identifiers, and augmented copies of an image could sit in both the training and test sets.
- **Not done:** on-device inference, a load balancer, a usability study with health workers (the SUS kit was used only with a few remote testers, see section 4), testing on a physical low-end or Android phone, external validation, and tests with smartphone-lens images.

Reference:

Reference: Gulshan, V., Peng, L., Coram, M., Stumpe, M. C., Wu, D., Narayanaswamy, A., … Webster, D. R. (2016). Development and validation of a deep learning algorithm for detection of diabetic retinopathy in retinal fundus photographs. JAMA, 316(22), 2402–2410. https://doi.org/10.1001/jama.2016.17216